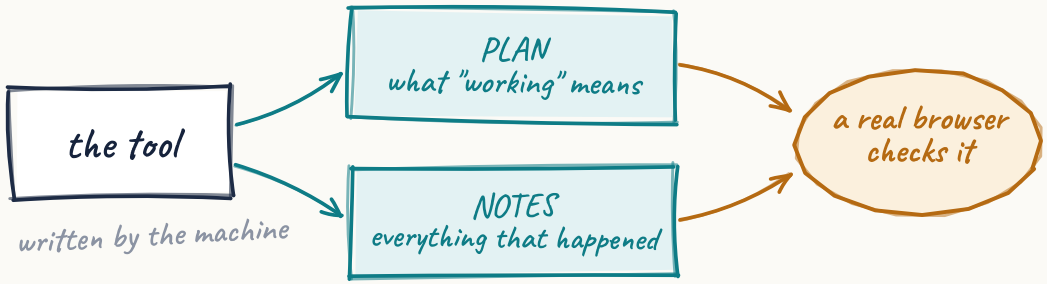
A WORKING MECHANISM · BUILT AND RUNNING IN PRODUCTION

Tools that carry their own documentation — and test themselves.

An autonomous website platform builds interactive tools without a human writing code. This is how each tool came to keep a living specification, a history written by the machines that change it, and a way to **prove in a real browser that it still works** — then file and check its own repair when it doesn't.



BUILT
4 – 16 July 2026

STATUS
Live · running unattended

EVIDENCE
Real screenshots & real records

"Deployed" is not the same as "works".

A generated tool can pass every structural check — valid HTML, a script block, nothing truncated — and still be broken in ways only a running browser reveals. A slider wired to the wrong variable. A chart line pinned to the wrong axis.

We watched exactly that ship **twice on the same game**: the original build introduced two behavioural bugs, and a later automated repair reproduced them faithfully. Every check we had passed, both times.

The pages photographed in this document are live sites you can visit: gamesdesign.co.uk and vonc.com.

THE PROBLEM

Two failure modes that kept biting

Both come from one root: nothing durable connected an artefact to its intent, or to a testable definition of “working”.

1 Lost intent

An agent builds a tool and makes deliberate choices — one reused input field instead of three; a raw canvas instead of a chart library. Weeks later another agent “improves” it and quietly undoes them, because nothing recorded *why*. The reasoning lived in a conversation that no longer exists.

2 “Deployed” ≠ “works”

Structural checks look at stored HTML; they cannot see behaviour. The simulator opposite passed all of them while two bugs sat in plain sight of anyone who used it — and an automated repair later put both bugs straight back.

What the simulator actually got wrong

- → The influx slider used its **position** (0–5) as the rate: “Flood” added five players a tick, not a hundred — about twenty times too weak to notice.
- → The players line was drawn on the **gold axis**, which autoscales to tens of thousands. Two hundred players rendered flat along the baseline.

Neither is visible in the HTML. Both are obvious in a browser.

gamesdesign.co.uk · the repaired simulator, live today. Three separate y-axes; the influx slider now maps its position to a real rate.

How it was put right

The repair went through the loop described in these pages: the tool was rebuilt from a specification naming both defects as requirements, and the fix was verified against the live page. The reasoning is on record in the tool's own notes, so the next agent inherits the argument rather than repeating it.

Both bugs were introduced by a machine. Both were documented and fixed by one.

IDEA ONE

Travelling docs: a PLAN and a NOTES log, keyed to the tool

Two documents live in the database beside the thing they describe. The agents write them as a byproduct of building and fixing, and load them before touching anything.

The PLAN — what this tool is, and what “working” means for it

Aim, behaviour contract, delivery mechanism, deliberate decisions (“do not re-fix this”), and — the load-bearing part — **acceptance criteria**: a machine-readable list of checks that define working *for this specific tool*. Edits supersede the previous version rather than overwrite it, so the history of intent survives.

The NOTES stream — an append-only history

Every fix, diagnosis, dead end and verdict, tagged by category and stamped with who wrote it. Dead ends matter as much as successes: a recorded “this was not the cause” stops the next agent re-running the same investigation.

Why a database, not files in a repo

- → Keyed by **function**, not by path — the docs survive the tool being copied to another site.
- → Appending a note is one insert. Appending to a shared file is a read-modify-write, and concurrent agents lose updates.
- → **Docs can never fail the work they document.** Every documentation step routes its errors to “done”: a tool is still built if its PLAN write fails.

doc_plans · tool-xp-curve-designer · current version

```
# PLAN — tool-xp-curve-designer

## Deliberate decisions — do not re-fix
- The growth-factor input swaps min/max/step/default per
  curve type: one reused field, not three — v1 kept simple
  by design.
- Raw 2D canvas, not a charting library — no external
  dependencies.
- Chart dots suppressed above 25 levels: a deliberate v1
  threshold.

## Acceptance criteria
```criteria
{ "profiles": ["desktop","mobile"],
  "checks": [
    {"id":"boots","type":"selector_exists",
     "selector":".tool-container"},
    {"id":"console","type":"no_console_errors"},
    {"id":"status","type":"page_status_ok"},
    {"id":"mobile-fit","type":"no_horizontal_overflow",
     "profiles":["mobile"]},
    {"id":"curve-switch","type":"interaction",
     "steps":[{"action":"select","selector":"#curveType",
               "value":"exponential"}],
     "expect":{"selector":"#tableWrap tr"}}
  ] }
```

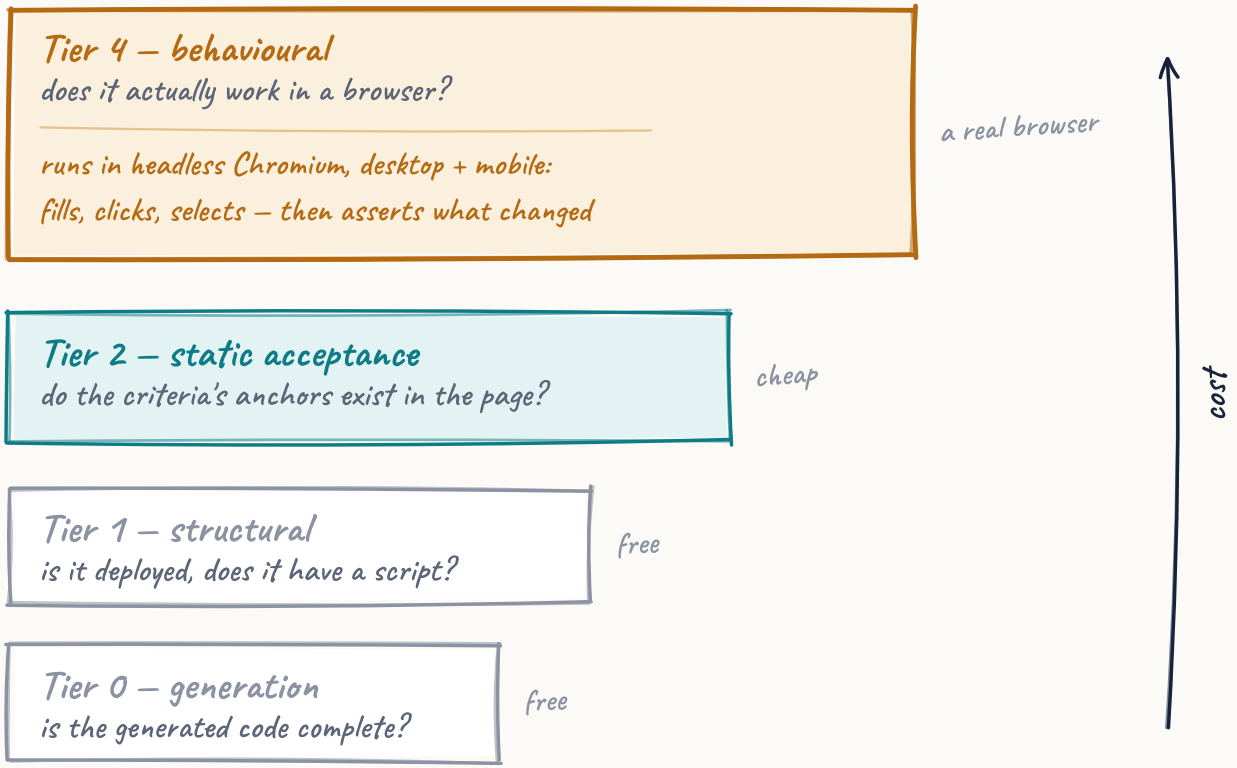
Correction log
- v1 (machine-written) asserted a table-body id the
  component never defines. Real: rows are rendered by
  script into #tableWrap.
```

Not an illustration — this is the live record, copied from the database today. The tool wrote the first version unaided, as it was created. The last block is the machine's own correction log.

IDEA TWO

A verification ladder: cheap checks confirm, the browser proves

Criteria are only worth writing if something checks them. Four tiers, cheap to expensive, each catching a class of problem the one below it cannot see.



The anchor rule

A criterion says "assert `#tableWrap tr` exists". The rows are built by JavaScript at runtime, so they are *not* in the stored HTML — but the container `#tableWrap` is.

So the static tier validates only the **anchor** — the leftmost id or class. A real-but-empty container passes and leaves the rows to the browser tier. A selector that exists nowhere fails.

Static checks confirm. They never refute.

Why that rule exists

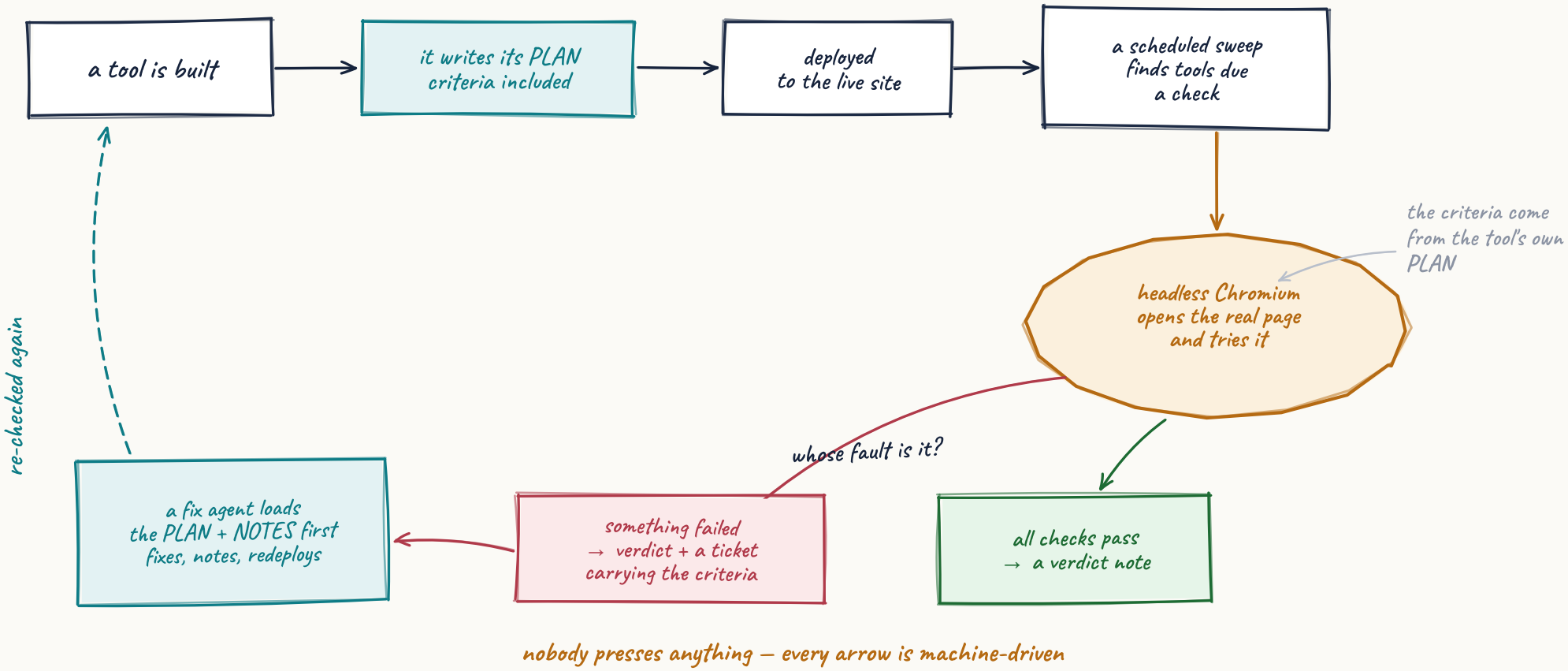
An agent writing a PLAN invented two element ids that were never in the tool. Without the anchor rule the cheap tier would either miss that, or delete legitimate checks for anything a page builds at runtime — which is most of them.

It has caught invented selectors **three times** — once where an agent wrote `#drop-chance` and the generator had emitted `#dropChance`.

HOW IT FITS TOGETHER

The loop — every arrow is machine-driven

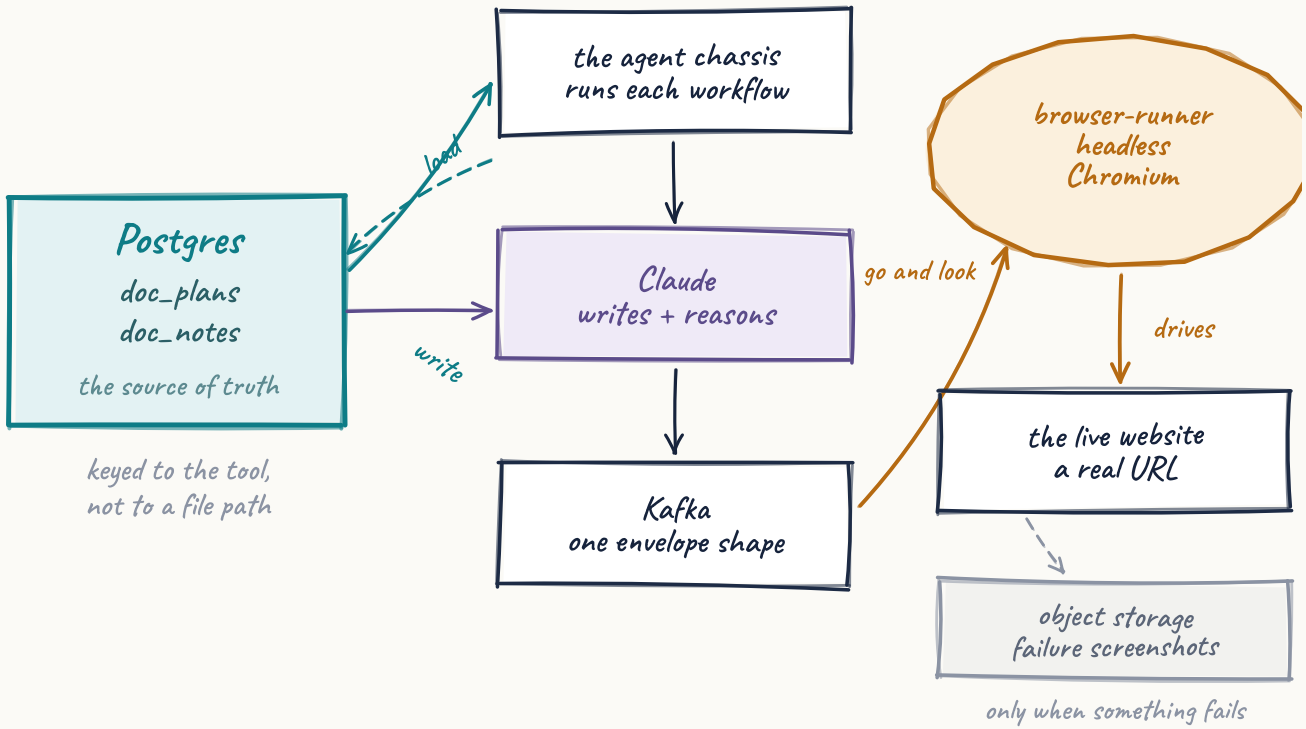
Nobody triggers any of this. A scheduled sweep finds tools due for a check; the verdict lands back in the same documents the criteria came from.



UNDER THE BONNET

Ordinary parts, arranged so the documents are the contract

Nothing here is exotic. The interesting decision is *where the truth lives*: in Postgres, keyed to the tool, written by the agents, read by anything that wants to know what “working” means.



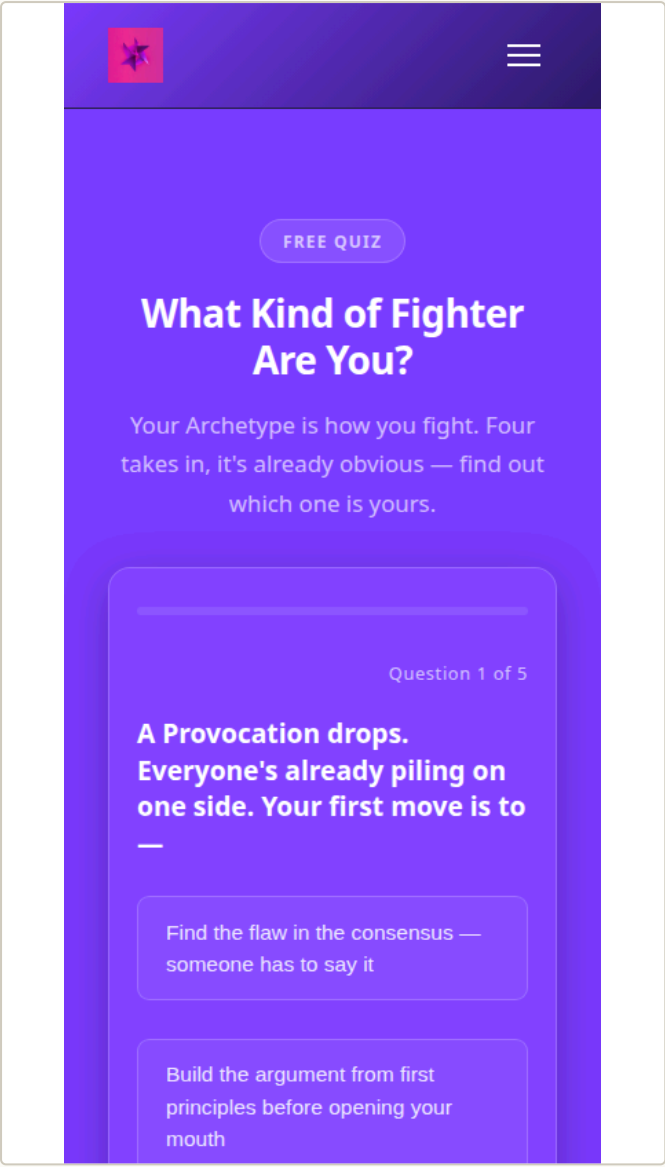
What each piece does

- → **Postgres** holds the PLAN and NOTES. It is the source of truth; everything else is derived.
- → **Kafka** carries requests between agents and adapters — one envelope shape, everywhere.
- → **The agent chassis** (Go) runs each agent's workflow: load docs, do the work, write a note.
- → **The browser-runner** launches headless Chromium, drives the live page, and reports what it saw.
- → **Claude** does the writing and reasoning: Sonnet 5 across the tool pipeline, Opus 4.8 for full rebuilds.

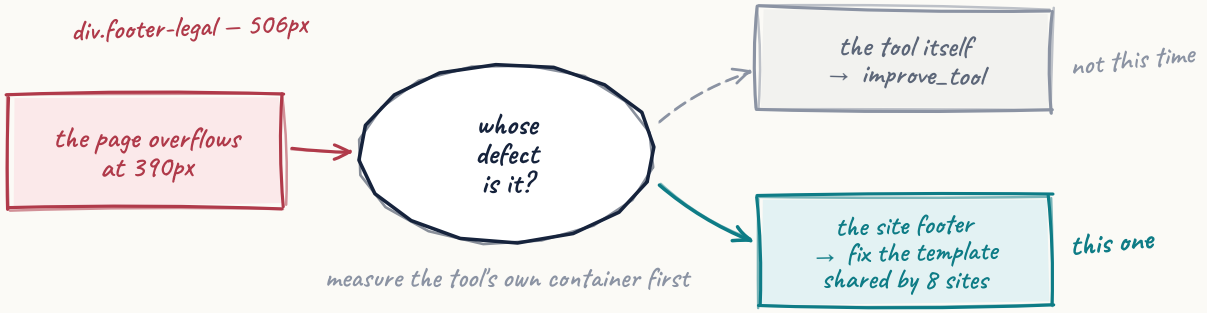
The rule that makes it safe

Every documentation step is wired so its failure routes to “finished”, not “failed”. Writing a note is never allowed to break the build it describes.

A quiz that was blameless, a footer that wasn't



vonc.com · 390px, as the browser tier sees it. The tool was fine. Something else was 506px wide.



1 · Found, on mobile only

The first genuine (not manufactured) failure. Three of seven checks failed. One was a stale criterion — the PLAN asserted a class this tool never had. The other was real: the page overflowed at 390px.

2 · Attributed, not assumed

Overflow is measured on the whole document, so the naive move — blame the tool — would raise an unfixable ticket against *every* tool on the site, forever. Instead the runner locates the tool's own container, names the widest offender, and labels the result **tool** or **site chrome**.

3 · The first fixer lied

The repair agent reported `"fixed": true` and changed nothing in the footer — it had defaulted to patching the *header*. We only knew because the browser re-measured and found the same 506px. **"Complete" is not "fixed", and only a behavioural check can tell them apart.**

4 · Fixed at the durable layer, then re-verified

The offender was one CSS rule in a footer template **shared by eight sites**. Patching the rendered page would have been erased on the next refresh, so the fix went into the template. The next run reported `mobile-fit@mobile` passing.

WHAT THE MACHINE WROTE

The verdicts, in the tool's own notes

Four consecutive entries from one tool's stream, unedited. Read top to bottom, they are the whole loop: a failure, a correction, a pass that still names the real culprit, and finally a clean run.

doc_notes · tool-archetype-taster-quiz · 14–15 July 2026

```
--- 14 Jul 10:34 | ["acceptance-fail"] | tool-acceptance ---
## Tier-4 acceptance FAILED — tool-archetype-taster-quiz
Observed: 3 of 7 evaluated checks failed in headless Chromium:
  boots: no element matches .tool-container in the live DOM
  mobile-fit: page overflows horizontally on mobile
Fix: improve_tool item created carrying the criteria as
  acceptance_test

--- 14 Jul 13:24 | ["migration"] | human ---
## PLAN superseded: real selectors + attribution
Root cause: composer-invented anchor + a PLAN born before the
  composer was fixed.
Fix: boots re-anchored to the REAL section class; quiz-flow
  replaced with a real interaction (click .quiz-option-btn x3
  -> .result-archetype-name) VERIFIED passing in real Chromium
  before being written here.
NOT fixed here: the page overflows on mobile — the offender is
  div.footer-legal (506px at 390px) in vonc's SITE FOOTER, which
  overflows every page on the site. That is site chrome, not this
  tool: routed to component-template-fixer. Blaming the tool
  would have sent the fixer to edit a component that cannot
  reach the footer.

--- 14 Jul 14:06 | ["acceptance-run"] | tool-acceptance ---
## Tier-4 acceptance PASSED — tool-archetype-taster-quiz
Observed: all 8 of the tool's own checks passed across profiles:
  desktop, mobile
Root cause: site chrome, not this tool: mobile-fit@mobile —
  div.footer-legal (479px) in site-footer
Fix: 1 responsive_fix item raised for the site template;
  the tool itself needs no change

--- 15 Jul 15:13 | ["acceptance-run"] | tool-acceptance ---
## Tier-4 acceptance PASSED — tool-archetype-taster-quiz
Observed: all 9 of the tool's own checks passed across
  profiles: desktop, mobile
Root cause: not-applicable
Fix: none required
Verified: boots@desktop, status@desktop, quiz-flow@desktop,
  console@desktop, boots@mobile, status@mobile,
  mobile-fit@mobile, quiz-flow@mobile, console@mobile
```

Read the third entry again

It records a **pass** and, in the same breath, names a defect it refuses to blame on this tool — with the exact element, its width, and where it lives. That is the difference between a test result and a diagnosis.

Why the last line matters

Between entries three and four, one CSS rule changed in a shared template. `mobile-fit@mobile` moves into the verified list on its own, because the criteria never changed — only the page did.

This is what a regression test looks like when the specification, the test and the history are the same document.

An honest habit worth stealing

“Verified in real Chromium *before* being written here.” Criteria are never authored from imagination — a check is watched passing against the live page first. A test you have never seen pass is a guess.

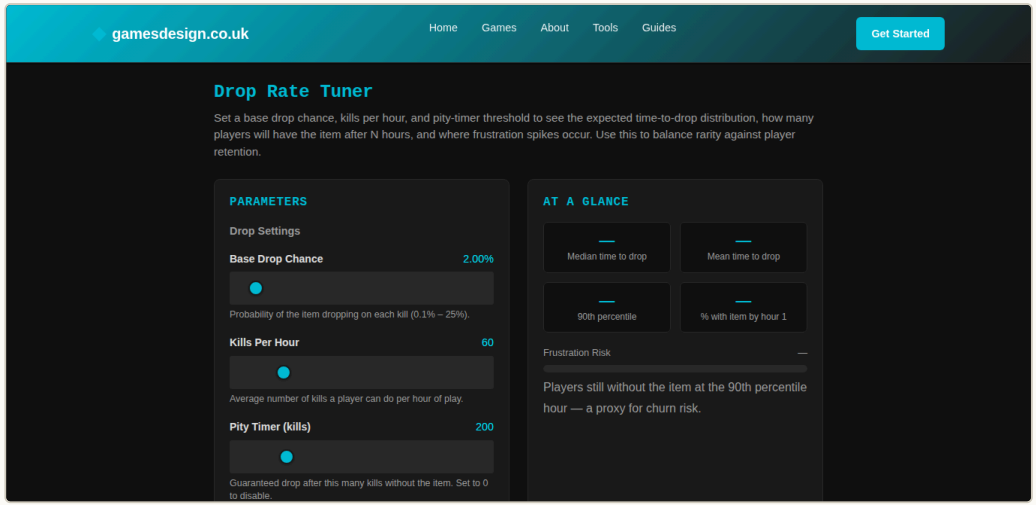
THE NEWEST LAYER

When a check fails, the page is photographed

A failing run photographs the whole live page, per profile, *while the browser is still open* — so the shot carries the state after the interaction that broke, not a fresh reload. The note gets a durable reference; the repair ticket gets a link a human can click.

Two rules that came out of building it

- → **Evidence is never load-bearing.** No storage configured, capture failed, upload failed — one log line, and the verdict is untouched. A page that never loaded is not photographed: there is no state worth keeping.
- → **Expiring links never enter the notes.** Notes are loaded into model prompts; a signed URL is hundreds of characters of soon-to-be-wrong signature. The note carries the durable address only, and the ticket carries the clickable one. There is a test pinning that.



gamesdesign.co.uk · the tool used for that proof. A deliberately-failing check was added to its PLAN, the run photographed the page at both sizes, and everything was reverted afterwards — the PLAN restored byte-for-byte, the manufactured notes deleted. Proving a mechanism should leave no litter.

It paid for itself on its first run

The proof run stored its screenshots correctly — and then no repair ticket appeared. That absence was a real bug: a cancelled ticket from days earlier still occupied the de-duplication slot, so every future ticket for that defect was being silently dropped. It had been swallowing tickets quietly for days. A one-line index fix closed it.

Looking closely at a pass is how you find what the pass was hiding.

IF YOU ARE BUILDING WITH AGENTS

Five things worth taking, whatever you are building

None of these need our platform. They are cheap, and each one came from something going wrong.

1 Give memory an address

Not “notes about the project” — notes keyed to the **thing**, loaded automatically whenever an agent touches it. Context recalled by luck is context you don't have. And record the dead ends: an agent that knows what wasn't the cause won't spend an hour rediscovering it.

2 Make “working” machine-readable, per artefact

A global test suite cannot say what *this* generated tool should do. Have the builder write the acceptance criteria at the moment it builds — it is the only point where the intent still exists. Keep them where the fixer will read them.

3 Check behaviour, not structure

Structural checks are free and worth having, but they cannot see a slider that does nothing. One headless browser turns “it looks right” into “it did the thing”. It is the tier that catches an agent reporting success it did not achieve.

4 Attribute before you route

A failure detected in one place is not a failure caused there. Work out *whose* defect it is before filing the ticket, or you will send a fixer to edit something it cannot reach — repeatedly, forever, and it will keep reporting success.

5 Never let the paperwork fail the work

If writing a note can break a build, someone will remove the notes. Route every documentation step's errors to “done”. The same applies to evidence: nice to have, never in the critical path.

And one about criteria

Criteria describe what you deliver, not what you meant to deliver. Ours asserted that JavaScript shipped as separate files, because a plan once said so. The pipeline had always inlined it. Every run failed, correctly, on a check that was wrong. Aspirations belong in a roadmap; a check that always fails just teaches everyone to ignore checks.

BEING STRAIGHT ABOUT IT

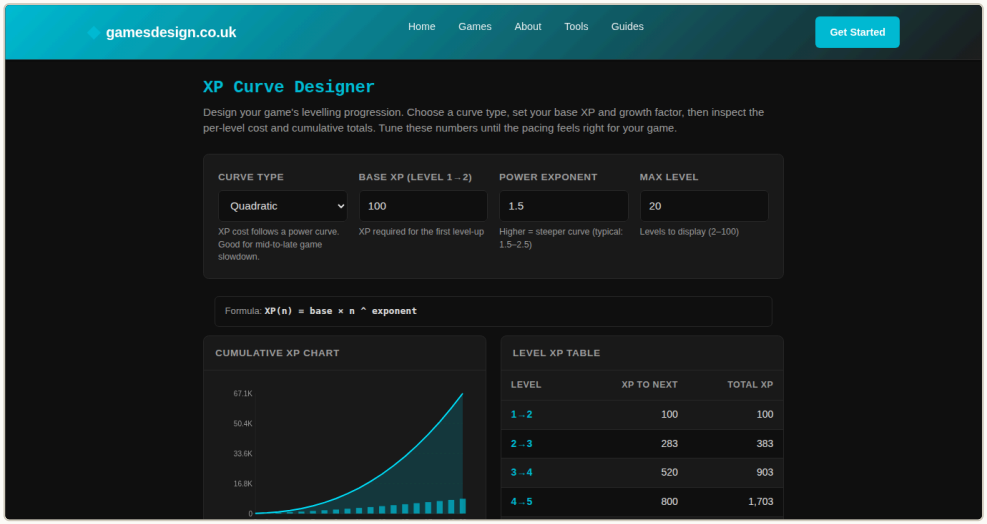
What this is, and what it isn't, today

Proven, in production

- → Tools write their own PLAN, criteria included, at the moment they are created.
- → Fix agents and the diagnosis loop append their own notes.
- → A scheduled sweep finds tools due a check and drives them in headless Chromium — desktop and mobile, with real interactions — with nobody triggering it.
- → Both outcomes work: a pass writes a verdict; a failure writes a verdict *and* files a repair ticket carrying the criteria.
- → It closed a real bug end to end, on a shared template across eight sites, and re-verified the fix itself.

Honest limits

- → It covers **tools** — the interactive, self-contained things. Not every page on every site.
- → The criteria are only as good as the agent that wrote them. Three invented selectors so far; the anchor rule caught all three, which is the point, but the writing still needs checking.
- → A failure arriving **naturally** with photographic evidence hasn't happened yet — the mechanism is proven, deliberately, on a bug we caused on purpose and then reverted.
- → Rolling the same discipline across the rest of the fleet is a decision, not a deployment.



The first tool that wrote its own PLAN — gamesdesign.co.uk, live. The table on the right is built by JavaScript; nothing static can see it. The browser tier counts twenty rows in the live DOM, then selects "exponential" and checks they rebuild. That single assertion is why the whole ladder exists.

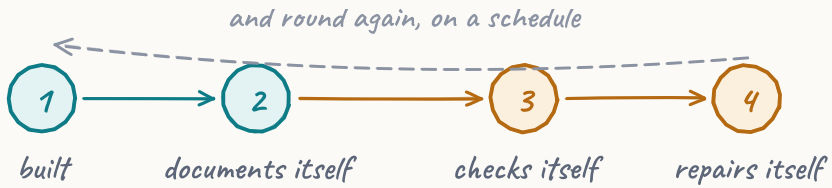
The bit that surprised us

Most of the value did not come from tools fixing themselves. It came from the machine being **forced to write down what it believed** — because then we could see it was wrong. Invented selectors, a delivery mechanism that never existed, a repair agent reporting success it hadn't achieved: each was invisible until something made a claim that could be checked.

IN ONE PARAGRAPH

A tool that says what it is for, states what “working” means, checks itself in a real browser, works out who is to blame, and writes down what happened.

The documentation travels with the thing it documents. The machines that change the thing are the ones that write its history — which is the only arrangement where documentation stays true. And the definition of “working” is not a paragraph anyone can interpret generously: it is a list of checks, driven in Chromium against the live page, that either pass or don't.



Part of an autonomous platform that plans, builds, deploys and maintains websites end to end. This document describes one mechanism inside it — the one that decides whether the work was any good.
Every screenshot is a live page. Every record shown is copied from the running database, unedited.

THE SHAPE OF IT, ONE MORE TIME

A tool is built.

It writes down what it is for, and what working means.

It goes live.

Something schedules a check. Nobody presses anything.

A real browser opens the real page and tries it.

The verdict is written where the next agent will read it.

If it failed: whose fault, which ticket, which fixer.

The fixer reads the history before it touches anything.

Then it is checked again.

The hard part was never the browser.
It was deciding what the tool
was allowed to promise.